

Detecting Machine-Generated Code under Multi-Lingual, Multi-Generator, and Adversarial Settings

Sohail Memon, Kathan Patel & Dhruv Rajput

International Institute of Information Technology – Hyderabad, India

{sohail.memon, kathan.patel, dhruv.rajpud}@students.iiit.ac.in

Abstract

The widespread adoption of large language models (LLMs) for source code generation poses significant challenges for reliable code authorship detection in education, hiring, and competitive programming. We present a comparative evaluation of five detection approaches for SemEval-2026 Task 13, encompassing binary detection (Subtask A), LLM family attribution (Subtask B, 11 classes), and hybrid/adversarial classification (Subtask C, 4 classes) under multi-lingual and multi-generator conditions. Our approaches span gradient-boosted trees with handcrafted features (CatBoost, XGBoost+NN), fine-tuned CodeBERT, a hybrid StarCoder2-3B architecture with 102 stylometric features, and a Semantic Graph Transformer operating on Abstract Syntax Trees with CodeBERT node embeddings. On the Kaggle competition leaderboard, the StarCoder2-3B hybrid achieves the best scores (**0.8938/0.2900/0.5559** Macro F1 on Tasks A/B/C), while classical ML with 230 handcrafted features reaches **98.23%** validation Macro F1 on Task A without pre-trained models. We find that data quality dominates model architecture, with a 58-point improvement from dataset curation alone, and that distribution shift between validation and test sets remains the primary challenge across all approaches.

1 Introduction

Large language models have recently demonstrated strong capabilities in automatic source code generation, enabling rapid software development and widespread access to programming assistance. However, this progress introduces serious concerns regarding academic integrity, fair hiring practices, and fairness in competitive programming environments. Detecting machine-generated code is substantially more difficult than traditional plagiarism detection, as LLM-generated code is often surface-level novel and structurally diverse.

Recent studies indicate that existing detectors often fail when evaluated under realistic distribution shifts, such as unseen programming languages, new generator families, or mixed human–AI code. These limitations significantly hinder the deployment of detection systems in real-world, integrity-critical settings. SemEval-2026 Task 13 directly addresses these challenges by providing a large-scale benchmark for evaluating detection systems under multi-lingual, multi-generator, and multi-domain conditions.

In this work, we implement and evaluate five detection approaches across all three subtasks, comparing gradient-boosted trees with handcrafted features, transformer-based fine-tuning, hybrid neural-feature architectures, and graph neural networks operating on code ASTs. Our experiments demonstrate that the hybrid StarCoder2-3B model achieves the best competition scores, while lightweight feature-based approaches offer compelling efficiency-performance trade-offs on same-distribution data but fail under distribution shift.

2 Related Work

Research on detecting machine-generated artifacts has grown rapidly with the emergence of large language models (LLMs). Existing work spans three main directions: (1) detection of machine-generated text, (2) detection of machine-generated source code, and (3) adversarial evasion of detection systems.

AI-Generated Text Detection. Early work on detecting synthetic content focused primarily on natural language text. Benchmarks such as M4 introduced multilingual and multi-generator datasets for evaluating AI-generated text detection across domains (Wang, 2024). Similarly, MULTITuDE provides a large-scale multilingual benchmark containing human and machine-written texts across multiple languages (Macko et al., 2023). Other

datasets such as MAGE evaluate robustness under domain shift and real-world deployment conditions (Li et al., 2024). In parallel, several shared tasks and competitions have advanced this area, including RuATD and other workshops on machine-generated content detection. While these efforts significantly improved text-based detection methods, studies have shown that detectors designed for natural language often fail when applied directly to source code due to fundamental structural differences between code and natural language (Wang et al., 2024). These findings motivate the development of specialized approaches for detecting machine-generated code.

AI-Generated Code Detection. Initial approaches to identifying AI-generated code relied on stylometric features and traditional machine learning models. For example, Idialu et al. (2024) explored decision-tree-based classifiers using code statistics such as Abstract Syntax Tree (AST) depth, identifier lengths, and code formatting patterns. Similarly, Li et al. (2023a) demonstrated that handcrafted features capturing code structure and lexical statistics can effectively distinguish human-written and ChatGPT-generated code.

More recent work has shifted toward deep learning approaches. GPTSniffer (Nguyen et al., 2024) and GPT-Sensor (Xu, 2025) train transformer-based models on pairs of human-written and machine-generated functions derived from the CodeSearchNet corpus (Husain et al., 2019). These methods treat detection as a supervised classification problem and learn discriminative representations of code. However, many early datasets rely heavily on CodeSearchNet, limiting diversity in programming languages, code domains, and generator models.

CoDet-M4 demonstrates that pretrained code models significantly outperform traditional baselines in multilingual and multi-generator detection settings, highlighting the importance of diverse training data (Orel et al., 2025a).

A major recent contribution is Droid (Orel et al., 2025b), which introduces DroidCollection, one of the largest datasets for machine-generated code detection. It contains over one million code samples across seven programming languages and outputs from 43 different code-generation models. The dataset includes not only purely AI-generated code but also hybrid human-AI collaborations and adversarially modified code designed to evade detection.

Experimental results show that many existing detectors struggle to generalize across languages and domains, highlighting the importance of large and diverse training datasets.

Adversarial Evasion and Robust Detection. Another emerging line of work investigates adversarial attacks on machine-generated content detectors. Prior studies demonstrate that detectors trained on straightforward AI-generated outputs can be easily bypassed using paraphrasing, rewriting, or stylistic modifications (He et al., 2024). Benchmarks such as RAID explore adversarial evaluation for text detection systems by applying perturbation strategies like synonym substitution and paraphrasing (Dugan et al., 2024).

Pre-trained Code Models. Recent progress in code intelligence has been driven by large pre-trained models trained on massive code corpora. CodeBERT (Feng et al., 2020) introduced bimodal pre-training on code and natural language pairs, enabling strong performance on multiple code understanding tasks. Subsequent models such as StarCoder (Li et al., 2023b) and StarCoder2 (Lozhkov et al., 2024) scale this paradigm to larger datasets and model sizes. These models learn rich semantic representations of programming languages, making them suitable for downstream tasks including code classification, vulnerability detection, and authorship attribution.

3 Task Description

SemEval-2026 Task 13 evaluates machine-generated code detection across three subtasks: (A) binary classification of human versus machine-generated code, (B) multi-class attribution to specific LLM generator families (11 classes), and (C) fine-grained classification including hybrid and adversarial code (4 classes). This work addresses all three subtasks, with three approaches (CodeBERT, StarCoder2-3B, Semantic Graph Transformer) submitted to the Kaggle leaderboard for all subtasks and two feature-based baselines (CatBoost, XGBoost+NN) evaluated on Task A.

The task includes distribution shifts across programming languages, generators, and coding domains, reflecting realistic scenarios such as LLM-assisted academic submissions and competitive programming. The official evaluation metric is macro F1-score. The task definition follows the official benchmark (SemEval-2026 Task 13 Organizers,

Language	Train	Train (%)	Val	Val (%)
Python	457,306	91.46	91,461	91.46
C++	23,392	4.68	4,679	4.68
Java	19,302	3.86	3,860	3.86

Table 1: Language distribution for Task A.

2026).

4 Dataset

We use the official SemEval-2026 Task 13 dataset from HuggingFace (SemEval-2026 Task 13 Organizers, 2026). For Subtask A, the training set contains 500K algorithmic code snippets in C++, Python, and Java, with 238K human-written (47.7%) and 262K machine-generated (52.3%) samples. The validation set comprises 100K samples with similar class distribution.

The dataset is designed to evaluate generalization under distribution shift. Evaluation settings include: (1) seen languages from training, (2) unseen languages (Go, PHP, C#, C, JavaScript), (3) seen algorithmic domain, and (4) unseen domains (research and production code). This comprehensive evaluation protocol ensures that successful approaches must learn robust, transferable features rather than domain-specific artifacts.

Language Distribution (Task A). Table 1 shows the language distribution for Subtask A.

Label–Language Distribution (Task B). Table 2 presents the cross-tabulation of labels and languages for Subtask B.

4.1 Dataset Balancing

To mitigate class imbalance and ensure fair evaluation, we apply task-specific balancing strategies.

Task A. Instead of using the original Subtask A distribution, we construct a balanced dataset derived from Subtask B. Specifically, we merge all machine-generated classes into a single *AI* label and retain the *Human* class. This results in a more balanced distribution across programming languages, reducing the heavy skew toward Python observed in the original dataset.

Task B. Subtask B exhibits significant class imbalance, primarily due to the dominance of human-written samples. To address this, we adopt a hybrid sampling strategy: (i) for classes with fewer samples, we retain all available data, and (ii) for

classes with larger sample sizes (including the human class), we perform downsampling to match the scale of lower-resource classes. This ensures a more uniform distribution across all authorship classes while preserving diversity in underrepresented categories.

Task C. For Subtask C, we enforce strict class balance by uniformly sampling 85K instances from each class. This results in an equal representation of all categories, enabling unbiased evaluation of hybrid and adversarial detection.

Validation Set. We apply the same balancing strategy to the validation sets of all subtasks to maintain consistency between training and evaluation distributions.

Balanced Dataset (Memon et al., 2026a)

5 Methodology

We implement and evaluate five detection approaches for SemEval-2026 Task 13, addressing all three subtasks: binary detection (A), LLM family attribution (B), and hybrid/adversarial classification (C). Our approaches span gradient-boosted trees with handcrafted features, transformer-based fine-tuning, a hybrid frozen-backbone architecture, classical ML ensembles, and a graph neural network operating on code ASTs. Each approach is described below.

5.1 Approach 1: CatBoost with Handcrafted Features

Our first approach uses CatBoost gradient-boosted decision trees trained on 36 language-agnostic features extracted from raw code text via string operations and regular expressions, covering length statistics, whitespace/indentation patterns, lexical diversity, comment ratios, structural proxies (brace depth, function count), punctuation density, character/token entropy, and line repetition. CatBoost was trained with 500 iterations, learning rate 0.05, tree depth 6, automatic class weight balancing, and early stopping (patience 50) on all 500K training samples.

5.2 Approach 2: Fine-tuned CodeBERT

CodeBERT (Feng et al., 2020) (codebert-base, 125M params) is a RoBERTa-based transformer pre-trained bimodally on natural-language–code pairs from

Lang	Total	Human	DS	Qwen	01-ai	BigCode	Gemma	Phi	LLaMA	IBM	Mistral	OpenAI
Java	164,583	148,761	1,263	2,328	704	154	496	996	1,787	3,334	1,510	3,250
Python	163,917	139,821	1,061	3,840	1,697	2,172	451	3,812	4,594	2,575	1,215	2,679
C#	75,356	67,858	673	1,047	313	26	278	394	773	1,434	775	1,785
JavaScript	50,083	41,874	706	1,465	442	40	365	542	1,037	498	642	2,472
C++	43,956	37,301	352	1,313	315	262	216	822	1,054	566	417	1,338
Go	35,565	32,183	302	420	115	12	145	213	326	676	371	802
PHP	35,094	34,334	304	40	0	0	194	0	50	1	171	0
C	31,446	28,454	348	295	93	6	195	122	271	622	402	638

Table 2: Label $\times$ language distribution for Task B.

CodeSearchNet (Husain et al., 2019) using MLM and RTD objectives.

Architecture. A sequence-classification head is appended to the frozen-then-fine-tuned encoder. The [CLS] hidden state (768-d) passes through Dropout ($p=0.1$) and a linear layer projecting to k classes: $k=2$ (Task A), $k=11$ (Task B), $k=4$ (Task C). Cross-entropy with label smoothing ($\epsilon=0.1$) is used throughout.

Training Configuration. All models run on Kaggle 2 $\times$ T4 GPUs via HuggingFace Trainer (DataParallel). Data is streamed from our balanced HuggingFace Hub dataset ((Memon et al., 2026a)). EarlyStoppingCallback (patience 2) restores the best checkpoint automatically.

Table 3: Fine-tuning hyperparameters

Hyperparameter	Value
Max sequence length	256 tokens
Per-device batch size	32
Effective batch size	64 (2 GPUs)
Learning rate	1×10^{-5}
LR scheduler	Cosine decay
Warmup ratio	10% of steps
Max epochs	5 (early stopping)
Weight decay	0.05
Label smoothing	0.1
Mixed precision	FP16
Eval frequency	Every 200 steps

5.3 Approach 3: Hybrid StarCoder2-3B with Handcrafted Features

This approach combines a frozen 3B-parameter StarCoder2 backbone with 102 handcrafted features in a hybrid architecture. The core insight is that a large pre-trained code model encodes deep

semantic representations, while handcrafted features capture surface-level stylistic patterns that are compressed or lost during sub-word tokenisation.

Frozen Backbone Design. bigcode/starcoder2-3b (3.03B parameters) is loaded with 8-bit quantisation using BitsAndBytesConfig, reducing GPU memory from ~ 12 GB to ~ 4 GB while preserving representation quality. All backbone parameters remain frozen; the model serves purely as a feature extractor. This offers three advantages: (1) training updates only ~ 8 M of the total ~ 3 B parameters ($\approx 0.27\%$), preventing catastrophic forgetting; (2) backbone embeddings are computed *once* per split and cached as .pt files, so every training epoch processes only the lightweight fusion head; and (3) 8-bit quantisation makes the 3B backbone feasible on a single consumer-grade GPU.

Architecture. Figure ?? summarises the data flow. Given a code snippet, tokens are extracted with the StarCoder2 tokeniser (max 512 tokens). The frozen encoder produces per-token hidden states of dimension 3072 (StarCoder2-3B hidden size). **Masked mean pooling** averages over non-padding positions, yielding a single 3072-d representation. A trainable *pooler* (Linear $3072 \rightarrow 3072 + \tanh$) projects this into the fusion space.

In parallel, a *Feature MLP* processes the 102-d handcrafted feature vector through two fully-connected layers ($102 \rightarrow 256 \rightarrow 128$) with LayerNorm, GELU activation, and 10% dropout. The pooler output and the Feature MLP output are concatenated to form a 3200-d joint representation ($3072 + 128 = 3200$), which is passed through a *Fusion MLP*:

$$3200 \xrightarrow{\text{LN-GELU-Drop}_{0.2}} 512 \xrightarrow{\text{LN-GELU-Drop}_{0.1}} 256 \xrightarrow{\text{Linear}} K$$

where $K \in \{2, 11, 4\}$ for Tasks A, B, and C respectively.

Handcrafted Features (102 dimensions). The feature set complements neural representations with three groups:

AST Features (33): Tree depth, breadth, and balance metrics; node-type entropy and unique node ratios; cyclomatic complexity and nesting depth; function counts, average lengths, and parameter statistics; class metrics including method density and documentation presence.

Pattern Features (57): Naming-convention ratios (camelCase, snake_case, PascalCase); comment density for inline, block, and documentation styles; import organisation and standard-library ratios; syntactic construct frequencies (list comprehensions, lambda expressions); code-organisation metrics; and LLM-signature pattern counts that capture recurring idioms specific to particular generator families.

Stylometric Features (12): Line-length statistics, indentation depth and consistency, blank-line ratio, and comment-to-code density.

Training Strategy. Embeddings and handcrafted features are pre-extracted and cached for all splits (train, validation, test) before training begins. The fusion head is trained with AdamW ($\text{lr} = 5 \times 10^{-6}$, $\text{weight decay} = 0.01$) using CrossEntropyLoss with *inverse-frequency class weights*: each class c receives weight $w_c = N / (K \cdot n_c)$, mean-normalised so that the average weight equals 1. This directly addresses the severe class imbalance in Task B (88% human) and Task C. Training auto-resumes from the latest checkpoint; the best model is selected by validation macro F1.

5.4 Approach 4: Classical ML with 230 Handcrafted Features

As a strong baseline without pre-trained neural models, we develop a comprehensive feature engineering pipeline using only Python standard libraries (ast, re, collections) and train classical ML models.

Feature Engineering (230 dimensions): The feature set is organized into five complementary categories:

AST Features (40): Comprehensive tree metrics including maximum and average depth, breadth, and balance factor; node type distributions and entropy; control flow analysis (if/for/while/try counts, nested loop depth); function metrics (count, length, parameters, recursion detection); class analysis (inheritance depth, decorator usage, method counts).

Pattern Features (60): Naming convention detection for CamelCase, snake_case, and mixed styles with length statistics; comment pattern analysis including docstrings, inline comments, and TODO markers; code structure metrics (indentation consistency, line length distributions, blank line ratios); programming idiom detection (list comprehensions, f-strings, walrus operator usage); import pattern analysis.

Stylometric Features (30): Token-level statistics including vocabulary size and type-token ratio; character distribution analysis (alphanumeric ratios, special character frequencies); syntactic patterns (punctuation usage, operator frequency); readability metrics and complexity indicators; formatting consistency scores.

N-gram Features (80): Token n-grams (unigrams through trigrams) capturing keyword sequences and operator patterns; character n-grams (bigrams through 4-grams) encoding coding style fingerprints; TF-IDF weighting for discriminative n-gram importance; frequency analysis of common vs. rare code constructs.

Complexity Features (20): Cyclomatic complexity based on decision point counts; Halstead metrics (vocabulary, volume, difficulty, effort); maintainability index estimation; maximum and average nesting depth; code cohesion metrics.

Model Pipeline: We train two complementary models: XGBoost gradient-boosted trees (200 estimators, max depth 8, learning rate 0.1) that naturally handle varying feature scales, and a feed-forward neural network ($230 \rightarrow 512 \rightarrow 256 \rightarrow 128 \rightarrow 2$) with 30% dropout and class-balanced loss weights. Features are standardized for the neural network but kept raw for XGBoost. Final predictions combine both models through weighted probability averaging (0.5 each), leveraging the complementary strengths of tree-based and neural approaches.

5.5 Approach 5: Semantic Graph Transformer (GNN)

While Approaches 1–4 treat code as flat text or extract features from it, this approach operates directly on Abstract Syntax Trees (ASTs). Given a code snippet, we parse it into an AST using Tree-Sitter (8 languages) and convert the AST into a directed graph with four edge types: parent→child, child→parent, sibling, and next-token. Each AST node is represented by three concatenated feature

groups: (1) a learned node-type embedding (384-dim, 8K vocabulary), (2) six normalized structural features, and (3) a frozen CodeBERT (Feng et al., 2020) embedding (768-dim→384-dim projection). This 774-dim feature is projected to 768-dim via a linear layer with layer normalization.

The graph encoder uses 5 TransformerConv layers (12 heads each) with learned edge-type embeddings (96-dim) as edge attributes, residual connections, and layer normalization. JumpingKnowledge (max) preserves multi-scale representations, and global attention pooling yields a 768-dim graph embedding concatenated with a 100-dim TF-IDF vector capturing surface-level lexical patterns. The fused 868-dim vector passes through an MLP classifier (868→768→384→ C). Training uses focal loss with inverse-frequency weights, label smoothing (0.1), OneCycleLR with 30% warmup, gradient accumulation (effective batch 1,024), mixed-precision, and data augmentation (15–25% of samples via identifier renaming, whitespace reformatting, and comment injection). The model (17M parameters) is trained separately per subtask.

This approach evolved from a GCN+GAT baseline (v1–v6) using only 6-dim structural features and two edge types, which peaked at 0.552 F1 on Task A and collapsed on Tasks B/C. Adding CodeBERT semantics, directional edge types, and TransformerConv resolved the collapse (Orel et al., 2025b).

6 Code and Model Links

Code (Memon et al., 2026b)

CodeBERT - Task A (Memon, 2026a)

CodeBERT - Task B (Memon, 2026b)

CodeBERT - Task C (Memon, 2026c)

StarCoder (Patel, 2026)

GNN (Rajput, 2026)

7 Experimental Results

We evaluate all five approaches on SemEval-2026 Task 13 using Macro F1 as the primary metric. Table 4 reports Kaggle competition leaderboard scores for submitted models; Table 5 provides validation-set performance across all approaches.

7.1 Overall Performance

StarCoder2-3B achieves the best competition scores on all three subtasks, followed closely by

CodeBERT. The Semantic Graph Transformer’s initial competition submission suffered from model collapse (§7.2.5), though later retraining achieved substantially higher validation F1. CatBoost and XGBoost+NN were developed for Task A only.

7.2 Individual Model Analysis

7.2.1 Approach 1: CatBoost (36 Features)

CatBoost achieves 0.9735 validation Macro F1 on Task A, demonstrating that a small set of stylometric features captures strong discriminative signals on same-distribution data. The top-5 features by importance are: `std_line_length` (18.2%), `char_entropy` (12.4%), `unique_token_ratio` (9.7%), `avg_identifier_length` (8.3%), and `comment_to_code_ratio` (6.8%). Machine-generated code exhibits more uniform line lengths, lower character entropy, and more consistent identifier naming.

However, CatBoost drops to 0.3270 on the competition test set—a catastrophic 65-point decline. The test set includes unseen programming languages (Go, PHP, C#, C, JavaScript) and unseen domains (research, production code) that 36 surface-level features cannot generalise to, exposing a fundamental limitation of lightweight feature-only approaches under distribution shift.

Strengths: Fastest training (<1 min), fully interpretable, no GPU required. **Weaknesses:** No cross-language generalisation; Task A only; features overfit to training distribution.

7.2.2 Approach 2: CodeBERT (Fine-tuned)

CodeBERT delivers consistent performance across all three subtasks, achieving validation Macro F1 of 91.32% (A), 49.30% (B), and 73.89% (C), with competition scores of 0.8515, 0.2898, and 0.4967 respectively. The bimodal pre-training on code–documentation pairs enables effective cross-language transfer.

Task A shows a mild 6-point validation–test gap, indicating reasonable generalisation to unseen languages. **Task B** is harder: 88% human-class dominance and stylistic overlap among rare LLM families (Gemma, BigCode, 01-ai) limit macro F1 to 49.30 on validation, dropping to 28.98 on test. **Task C** (73.89% val, 49.67% test) is more tractable thanks to its coarser 4-way split, though hybrid and adversarial categories remain difficult.

Strengths: Strong on all three subtasks; benefits from pre-trained code understanding; moder-

Table 4: Kaggle competition test Macro F1 for submitted models on SemEval-2026 Task 13 (all subtasks).

Approach	Trainable Params	Task A	Task B	Task C
StarCoder2-3B + Handcrafted (Appr. 3)	8M	0.8938	0.2900	0.5559
CodeBERT fine-tuned (Appr. 2)	125M	0.8515	0.2898	0.4967
Semantic Graph Transformer (Appr. 5)	17M	0.5547	0.0600	0.2387
CatBoost (Appr. 1)	—	0.3270	—	—

Table 5: Validation Macro F1 for all approaches. Dashes indicate the model was not trained on that subtask.

Approach	A	B	C
CatBoost (36 feat.)	0.9735	—	—
CodeBERT (fine-tuned)	0.9132	0.4930	0.7389
StarCoder2-3B Hybrid	—	0.5014	—
XGBoost+NN (230 feat.)	0.9823	—	—
SGT (retrained, Kaggle)	0.9681	0.2849	0.6518

ate compute (125M params, $2 \times T4$ GPUs). **Weaknesses:** Requires GPU fine-tuning; 256-token truncation limits context; persistent val–test gaps.

Quantitative Analysis

Table 6 shows Precision, Recall, Accuracy, and Macro F1 on the validation set, plus the official Kaggle (held-out test) score.

Table 6: CodeBERT results across sub-tasks (%)

Task	P	R	Acc	F1	Kaggle
A (Binary)	92.02	91.36	91.36	91.32	85.15
B (11-class)	52.57	49.59	54.81	49.30	28.98
C (4-class)	75.73	73.60	73.61	73.89	49.67

Task A achieves strong binary detection (F1 91.32); the validation–test gap (85.15) suggests mild distribution shift on held-out data. **Task B** is harder: the 88% human-class dominance and stylistic overlap among rare LLM families (Gemma, BigCode, 01-ai) limit macro F1 to 49.30. **Task C** (73.89 F1) is more tractable thanks to its coarser 4-way split.

Qualitative Analysis

We examine representative *correct* and *misclassified* examples.

Task A — Binary Detection. Correct (Human, C++). Competitive-programming boilerplate with shorthand macros and no docstrings — correctly labelled *Human*:

Listing 1: Task A correct: human-written C++

```
#include <bits/stdc++.h>
#define nl << "\n"
using namespace std;
void removeLeading(string &str) {
    int idx = 0, n = str.size();
    while (idx < n && str[idx] == '0') idx++;
    str = str.substr(idx);
}
```

Misclassified (Machine → Human). A terse, idiomatic Go constructor without docstrings was incorrectly predicted as *Human*:

Listing 2: Task A error: AI-generated Go predicted as Human

```
func NewConfig(ginMode gin.Mode,
    cors, session bool, ...) *Config {
    return &Config{
        GinMode: ginMode, Cors: cors,
        Session: session,
    }
}
```

Analysis: The absence of boilerplate comments makes this indistinguishable from hand-written Go within the 256-token context.

Task B — LLM Family Attribution. Correct (IBM-Granite, C#). Formal namespace resolution with null-coalescing — correctly attributed to *ibm-granite*:

Listing 3: Task B correct: IBM-Granite generated C#

```
public class TypeNamespaceConverter {
    public string ConvertToPhpNamespace(
        Type t, Settings s) {
        string ns = t.Namespace;
        string pfx = s?.phpNamespacePrefix ?? "Beta";
        // ...
    }
}
```

Misclassified (Mistral → IBM-Granite). Clean minimal Java boilerplate is confused between the two families:

Listing 4: Task B error: Mistral predicted as IBM-Granite

```
public class SphereTrigger {
    private Entity entity;
    private Command command;
    public void onEnter() {
        if (command != null) command.execute();
    }
}
```

Analysis: Mistral and IBM-Granite share nearly identical Java formatting conventions; surface style

alone is insufficient to separate them.

Misclassified (Meta-LLaMA → BigCode). A minimal Java interface fragment is misattributed:

Listing 5: Task B error: Meta-LLaMA predicted as BigCode

```
package com.showka.service.query.u08.i;
import java.util.*;
public interface NyukinFBFurikomiQuery {
    Nyukin getNyukin(
        @PathVariable String fbFurikomiId);
}
```

Analysis: Both families produce sparse interface stubs with similar import patterns, making attribution ambiguous at 256 tokens.

Task C — Fine-grained Attribution. Task C classifies code as *human*, *machine*, *hybrid* (human-edited AI), or *adversarial* (evasion-crafted AI).

Correct (Human, Java). An AWS SDK utility with deprecation annotation — correctly labelled *human*:

Listing 6: Task C correct: human-written Java (AWS SDK)

```
@Deprecated
public static AWSXRay newClient() {
    DaemonConfiguration cfg =
        new DaemonConfiguration();
    ClientConfiguration cc =
        new ClientConfiguration()
            .withRequestTimeout(TIME_OUT);
    // ...
}
```

Misclassified (Machine → Hybrid, C#). A verbose `[Obsolete]` annotation resembling human-edited prose caused the error:

Listing 7: Task C error: Machine predicted as Hybrid

```
[Obsolete("Will be removed. Use alternate.")]
public static void EnableUser(
    this IElasticClient client,
    string username,
    CancellationToken ct = default) { }
```

Analysis: The documentation style mimics editorial human prose, crossing the machine/hybrid boundary. Adversarial samples (e.g. with commented-out debug prints and informal variable names) are similarly predicted as *human*, highlighting the difficulty of evasion detection.

7.2.3 Approach 3: Hybrid StarCoder2-3B

StarCoder2-3B achieves the best competition scores: 0.8938 (A), 0.2900 (B), 0.5559 (C). The critical finding is that *data quality dominates architecture*: the identical model scored only 0.3116 on Task A with raw 3-language data versus 0.8938 on a curated 8-language dataset—a 58-point improvement from data alone.

On Task B, validation reaches 0.5014 but test

drops to 0.2900, a 21-point gap caused by distribution shift between the balanced training set and the skewed test distribution. Task C shows consistent data-scaling behaviour: increasing training data from 50K to 150K samples improved test F1 from 0.5185 to 0.5559, suggesting the model has not saturated.

The frozen-backbone design enables rapid training (~2 min/epoch with only 8M trainable parameters) while preserving StarCoder2’s pre-trained representations. The 102 handcrafted features complement backbone embeddings by encoding surface-level stylistic signals that mean pooling may compress away. The model also exhibits asymmetric detection: higher recall for *Human* code (99.29%) versus *Machine* code (94.82%), suggesting conservative labelling when uncertain—a desirable property for academic integrity applications.

Strengths: Best test scores; only 0.27% trainable parameters; fast training; works across all sub-tasks without architectural changes. **Weaknesses:** Frozen backbone cannot adapt to novel patterns; mean pooling loses token-level structure; performance depends heavily on training data quality.

7.2.4 Approach 4: Classical ML (XGBoost + Neural Network)

The XGBoost+NN ensemble achieves 0.9823 validation Macro F1 on Task A—the highest same-distribution score among all approaches—without any pre-trained language models. The 230 features span AST metrics (40), pattern features (60), stylistic measures (30), n-gram encodings (80), and complexity indicators (20), providing comprehensive code style characterisation. Cross-validation (5-fold) yields $98.1\% \pm 0.3\%$ accuracy, confirming robustness across data splits.

This approach was not submitted to the competition leaderboard and was developed for Task A only. Given CatBoost’s catastrophic test-set drop with just 36 features, the 230-feature ensemble would likely also degrade under distribution shift, though its richer feature set—particularly n-gram and complexity features—may transfer somewhat better to unseen languages.

Strengths: Highest validation F1; no pre-trained models needed; interpretable features; low compute. **Weaknesses:** Untested on competition leaderboard; Task A only; probably vulnerable to distribution shift; AST features use Python `ast` module (limiting cross-language support).

Table 7: Semantic Graph Transformer validation Macro F1 after retraining on Kaggle T4 GPUs.

Subtask	Classes	Best Val F1	Best Epoch
A (Binary)	2	0.9681	47
B (Attribution)	11	0.2849	42
C (Hybrid)	4	0.6518	39

7.2.5 Approach 5: Semantic Graph Transformer

The initial competition submission suffered from model collapse, scoring 0.5547 (A), 0.0600 (B), and 0.2387 (C)—near or below random baselines for Tasks B/C. Root-cause analysis identified three compounding optimisation failures:

1. **Gradient accumulation scaling:** Loss division by 32 steps produced gradients too small to escape the majority-class attractor.
2. **Dual loss correction:** Simultaneous focal loss modulation and inverse-frequency class weights created unstable gradient magnitudes for minority classes.
3. **Aggressive LR decay:** OneCycleLR’s cosine schedule decayed the learning rate before the model could break out of majority-class prediction.

After retraining on Kaggle T4 GPUs with 50 epochs and patience 15, the model achieved substantially improved results (Table 7).

Task A reaches 0.9681 val F1, surpassing the GCN+GAT baseline (0.552) by over 40 absolute points and exceeding CodeBERT’s validation score (0.9132). This demonstrates that AST structure combined with CodeBERT node semantics and TF-IDF captures the essential signals for binary detection.

Task B remains challenging at 0.2849 val F1. The extreme class imbalance (88% human, $\leq 2\%$ per generator) and stylistic overlap among minority generators limit structural features’ discriminative power. AST-based methods inherently underperform text transformers on fine-grained attribution (Orel et al., 2025b).

Task C achieves 0.6518 val F1. The model handles human and machine classes well but confuses hybrid and adversarial categories, which deliberately blend authorship signals.

Evolution from GCN+GAT. The earlier baseline ($\sim 1.2\text{M}$ params) using 6-dim structural features and two edge types peaked at 0.552 Task A F1

and collapsed entirely on Tasks B/C. Transformer-Conv with CodeBERT embeddings and directional edge types resolved the collapse.

Strengths: Operates on code structure (theoretically robust to surface-level adversarial attacks); interpretable attention over AST nodes; language-agnostic graph construction. **Weaknesses:** Optimisation-fragile (initial collapse); cannot match text transformers on attribution; graph pooling discards local authorship boundaries; high compute ($\sim 17\text{M}$ params, $\sim 10\text{ GB GPU}$).

7.3 Cross-Model Comparison

Distribution Shift. Every submitted model exhibits a substantial validation–test gap: CatBoost (-65 pts), SGT (-41 pts), CodeBERT (-6 pts). The gap correlates inversely with the richness of pre-trained representations: models leveraging large-scale code pre-training (StarCoder2, CodeBERT) bridge distribution shift far better than feature-engineered or AST-based approaches.

Task Difficulty Gradient. Across all models, Task A $\gg$ Task C $\gg$ Task B. This hierarchy reflects increasing label granularity and worsening class imbalance. On Task B, no approach exceeds 0.29 test F1 (0.50 val for StarCoder2-3B), indicating that fine-grained generator attribution remains an open challenge.

Feature Engineering vs. Pre-training. On same-distribution validation data, handcrafted features are highly competitive: XGBoost+NN (0.9823) and CatBoost (0.9735) match or exceed neural models. Under distribution shift, pre-trained representations are essential. This motivates hybrid architectures like StarCoder2-3B that combine both paradigms.

Complementary Signal Types. Each approach captures different information: CatBoost and XGBoost+NN model surface-level stylometric patterns; CodeBERT captures token-level semantics; StarCoder2 encodes deep code understanding augmented with handcrafted features; the Semantic Graph Transformer exploits syntactic structure and code flow. An ensemble combining text-based and structure-based models could achieve stronger performance, as AST structure is invariant to surface-level adversarial modifications that may fool text classifiers.

Table 8: Cross-model comparison on key dimensions. Val and Test refer to Macro F1 on Task A.

Approach	Val A	Test A	Multi-task	Params	Key Trade-off
CatBoost (36 feat.)	0.9735	0.3270	No	<1K	Fast but no generalisation
CodeBERT (fine-tuned)	0.9132	0.8515	Yes	125M	Strong all-round; needs fine-tuning
StarCoder2-3B Hybrid	—	0.8938	Yes	8M	Best test scores; data-dependent
XGBoost+NN (230 feat.)	0.9823	—	No	<1K	Best val F1; untested OOD
Sem. Graph Transformer	0.9681	0.5547	Yes	17M	Structural insight; optim-fragile

8 Discussion

Error Analysis. Across all approaches, false negatives frequently involve simple utility functions—short, algorithmic code with minimal stylistic variation—and well-commented AI-generated code that mimics human documentation patterns. False positives often occur with templated human code (boilerplate, configuration files) that exhibits machine-like uniformity. On Task B, Mistral and IBM-Granite produce nearly identical Java formatting conventions, making surface style alone insufficient for separation. On Task C, adversarial samples containing commented-out debug prints and informal variable names are systematically misclassified as *human*, highlighting the difficulty of evasion detection.

Architecture Complexity $\neq$ Performance. The 17.4M-parameter Semantic Graph Transformer initially performed worse than the 1.2M-parameter GCN+GAT baseline, demonstrating that architectural sophistication without stable optimisation is counterproductive. Only after careful hyperparameter tuning did the richer architecture realise its potential, a finding consistent with the broader observation that optimisation dynamics (learning rate schedules, loss scaling, batch size) matter as much as model capacity for this task.

Data Quality as the Dominant Factor. StarCoder2-3B’s 58-point Task A improvement from data curation alone—without any model change—is the strongest single finding. Representative multi-language, multi-generator training data matters far more than model complexity for code authorship detection.

Limitations. Consumer hardware constraints (Kaggle T4 GPUs, RTX 4060 8GB) limited training scale for all approaches; StarCoder2-3B was

capped at 8-bit quantisation, and CodeBERT used DataParallel across $2 \times T4$. The XGBoost+NN and CatBoost approaches were developed for Task A only, leaving their multi-class potential unexplored. The Semantic Graph Transformer’s retrained validation scores have not been converted to competition test submissions due to time constraints; its true generalisation ability on unseen distributions remains to be verified.

9 Conclusion and Future Work

This work presents a comparative evaluation of five approaches for detecting machine-generated code across all three subtasks of SemEval-2026 Task 13: binary detection (A), LLM family attribution (B), and hybrid/adversarial classification (C). Our key findings are:

1. The hybrid StarCoder2-3B architecture achieves the best competition scores (0.8938 Task A, 0.2900 Task B, 0.5559 Task C) with only 0.27% trainable parameters, demonstrating that frozen code LLM backbones combined with handcrafted features offer an effective compute-efficiency trade-off.
2. Fine-tuned CodeBERT provides the most consistent performance across subtasks without requiring handcrafted features, achieving 0.8515/0.2898/0.4967 on the competition leaderboard.
3. Classical ML with comprehensive feature engineering (XGBoost+NN, 230 features) achieves the highest same-distribution validation F1 (0.9823 on Task A) without any pre-trained models, but handcrafted features do not generalise under distribution shift (CatBoost: 0.9735 val $\rightarrow$ 0.3270 test).

4. The Semantic Graph Transformer, operating on AST structure with CodeBERT node semantics, reaches 0.9681 validation F1 on Task A after resolving initial optimisation failures, validating the AST-based paradigm while exposing its sensitivity to training dynamics.
5. Data quality dominates model architecture: StarCoder2-3B’s 58-point Task A improvement from curated data underscores that representative, multi-language training sets are the highest-leverage intervention for code authorship detection.

References

- Liam Dugan and 1 others. 2024. Raid: A shared benchmark for robust evaluation of machine-generated text detectors. In *ACL*.
- Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. Codebert: A pre-trained model for programming and natural languages. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1536–1547.
- Xinlei He and 1 others. 2024. Mgtbench: Benchmarking machine-generated text detection. In *ACM CCS*.
- Hamel Husain and 1 others. 2019. Codesearchnet challenge: Evaluating the state of semantic code search.
- Oseremen Joy Idialu, Noble Saji Mathews, Rungroj Maipradit, Joanne M. Atlee, and Mei Nagappan. 2024. Whodunit: Classifying code as human authored or gpt-4 generated. In *Mining Software Repositories (MSR)*.
- Ke Li and 1 others. 2023a. Discriminating human-authored from chatgpt-generated code via discernable feature analysis. In *ISSRE Workshops*.
- Raymond Li and 1 others. 2023b. Starcoder: May the source be with you. *Transactions on Machine Learning Research*.
- Yafu Li and 1 others. 2024. Mage: Machine-generated text detection in the wild. In *ACL*.
- Anton Lozhkov and 1 others. 2024. Starcoder2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*.
- Dominik Macko and 1 others. 2023. Multitude: Large-scale multilingual machine-generated text detection benchmark. In *EMNLP*.
- Sohail Memon. 2026a. Codebert a model. https://huggingface.co/sohailtsm/semEval_taskA_model. Model.
- Sohail Memon. 2026b. Codebert b model. https://huggingface.co/sohailtsm/semEval_taskB_model. Model.
- Sohail Memon. 2026c. Codebert c model. https://huggingface.co/sohailtsm/semEval_taskC_model. Model.
- Sohail Memon, Kathan Patel, and Dhruv Rajput. 2026a. Balanced dataset. <https://huggingface.co/datasets/sohailtsm/my-semEval-dataset>. Dataset.
- Sohail Memon, Kathan Patel, and Dhruv Rajput. 2026b. Detecting machine-generated code: Implementation and experiments. <https://github.com/SohailTSM/Machine-Generated-Code-Detection>. GitHub repository.
- Phuong T. Nguyen and 1 others. 2024. Gptsniffer: Detecting ai-generated code using transformer-based models. In *Proceedings of ASE*.
- Daniil Orel, Dilshod Azizov, and Preslav Nakov. 2025a. CoDet-m4: Detecting machine-generated code in multi-lingual, multi-generator and multi-domain settings. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 10570–10593, Vienna, Austria. Association for Computational Linguistics.
- Daniil Orel, Indraneil Paul, Iryna Gurevych, and Preslav Nakov. 2025b. Droid: A resource suite for ai-generated code detection. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 31251–31277.
- Kathan Patel. 2026. Starcoder model. <https://huggingface.co/Kathan0/semEval2026-task13-models/tree/main>. Model.
- Dhruv Rajput. 2026. Gnn models. <https://huggingface.co/dhruv10050/semEval-gnn-models>. Model.
- SemEval-2026 Task 13 Organizers. 2026. Semeval-2026 task 13: Detecting machine-generated code with multiple programming languages, generators, and application scenarios. In *Proceedings of the International Workshop on Semantic Evaluation (SemEval-2026)*. Task description.
- et al. Wang. 2024. M4: A multilingual multi-generator benchmark for machine-generated text detection. *ACL*.
- Jian Wang and 1 others. 2024. An empirical study to evaluate aigc detectors on code content. *ASE*.
- et al. Xu. 2025. Gpt-sensor: Detecting ai-generated code via representation learning. *arXiv preprint*.